The Remaining Gap to Kissat Is Entirely Unit Cost: Ten Repairs, Six Refutations, and the Measurement That Settles It

Tactix development report for Andrew

5 September 2026

1 Objective and outcome

The objective is to close every implementation gap against Kissat: every case where Tactix does the same work and takes longer. Being within 10 % on Tier D Ranks 1–40 is the floor, not the finish. B28+C5 is the regression gate and N1–N4 are preserved.

Outcome. Ranks 1–40 stand at **8,535 s**, or **1.112**× Kissat's 7,675.1 s, from 8,781.1 s at the start of this work; B28+C5 improved from 0.495 to **0.463**. Ten repairs landed, each gated 40/40 valid with 10,000 fuzz cases and zero failures on both legs, and every one behaviour-preserving with identical solver counters. The net per-rule gap fell from roughly 500 s to **428 s**.

The goal is *not* met: 428 s of inprocessing gap remains. What this session can now say precisely, which it could not before, is what that 428 s is made of.

2 The result that matters

Every remaining rule gap is unit cost. Tactix performs *less* work than Kissat on every paired counter but one, while being slower on every rule but two.

rule	Tactix	Kissat	ratio	work counter	T/K
Substitute	282.2 s	169.6 s	1.66	variables substituted	1.00
Factor	237.8 s	129.8 s	1.83	ticks	1.03
Vivify	680.4 s	576.6 s	1.18	probes	0.83
Congruence	318.7 s	245.8 s	1.30	gates	1.03
BVE	260.0 s	194.1 s	1.34	resolutions	0.91
Strengthen	112.4 s	—	—	forward steps	0.88
TransitiveReduce	49.3 s	34.4 s	1.43		
Backbone	66.5 s	69.4 s	0.96		
Sweep	441.4 s	564.9 s	0.78	equivalences	0.98

Read the last two columns together. No rule can be closed by dosing it down, because none is doing surplus work; and no rule is buying its time with extra simplification. Strengthen has no honest phase counterpart, since Kissat subsumes inside `eliminate`: paired properly, BVE+Strengthen is 372.3 s against 230.2 s, a ratio of 1.62 and the largest single pairing left.

This is now reproducible in one command. `tools/bench/rule_gap_table.py` prints the table from a gate campaign plus a Kissat statistics baseline, and campaign `5064` supplies the latter — the pinned Kissat run under `-v` on all forty ranks, which is the only way it prints its full statistics. Phase times and counters come from different campaigns on purpose, since `-v` perturbs timing.

3 What was repaired

A diagnostic that shipped (170.6 s, the largest single repair). BVE ran an occurrence audit on every candidate elimination whose only consumers are trace calls that are empty inline functions in a production build.

Factor (281.8 s → 238 s). Per scanned entry Tactix took two random cache misses where Kissat takes one, because Kissat's skip mark is a clause header bit read from a line it already loads. Moving both marks there made the occurrence identifier dead, which let the watch record halve. Its remainder pool also moved from a map of bucket vectors to an open-addressed table.

Congruence's doubled ITE gates. Kissat canonicalises the triple so one gate covers a relation and its output-negated twin. Rank 22's gate count went from 10.48 M to 7.10 M against Kissat's 6.91 M with merges unchanged.

Five quadratic removals across substitution, the root-satisfied binary scans, the binary duplicate test and the dense occurrence lookup.

4 What was refuted

Six attempts measured neutral, slower, or unsound and were reverted rather than banked: Factor's local tick accumulation; guarding fourteen trace-only computations the compiler already eliminates; BVE's per-clause vector hoist; capping the binary copy count; bulk-stripping an eliminated variable's binaries (slower, and a first version was unsound); and deriving the large occurrence index from the ordered one.

Three structural hypotheses also died to measurement. Propagation is 42 % of runtime and looked 1.5× slower on Rank 31, but that instance runs 2.7× Kissat's conflicts; at equal counts the rates are within 9 %. Sweep, which an earlier report named the largest gap on one rank's evidence, is 124 s *faster* than Kissat tier-wide. And the claim that dense-mode rules pay to maintain sparse watches too is wrong: on Rank 21 that machinery is 0.05 % against Kissat's roughly 7 %.

5 Three measurement corrections

Each of these changed a conclusion, and they are the most transferable output of the session.

Allocation shape. Early A/B campaigns ran at four CPUs and 16 GiB, which the benchmark runner documents as the *debug* contract; the gates use twelve and 48 GiB. The two-word Factor watch record was refuted under those conditions and, retried correctly, proved worth 2.8 s.

The tier total cannot arbitrate. One repair measured a consistent −1.4 s in paired repeats and returned a gate **72.8 s slower** with all forty conflict counts identical. Its noise band is far wider than any single repair; paired same-node repeats must decide.

A patched reference that compiles can still be wrong. Instrumenting Kissat to expose its vivification propagations, I declared the propagate candidate `unsigned` where the real signature takes `struct clause *`. It compiled, ran, reported plausible counters — and answered SATISFIABLE on an UNSAT instance. Any patched reference must now match the pristine build on the verdict and one pre-existing counter before a new number is read from it.

6 Correctness

The proof and model checks of all ten gate pairs run in this session have finished: 500 proof checks and 560 model checks, every one exit code 0, with no instance whose verdict disagreed with its expected result. N2 holds for every commit landed, independently of the counter identity used to judge the repairs.

7 The tier gap and the implementation gap are different problems

Decomposing the Rank 1–40 total by rank, which should have happened far earlier, separates two things this project has discussed as one.

rank	Tactix	Kissat	gap	share	conflicts T/K
34	740.7 s	344.0 s	+396.6	46.1 %	1.97
31	673.3 s	277.3 s	+395.9	46.1 %	2.73
17	256.5 s	121.8 s	+134.7	15.7 %	1.16
all 40	8,535.2 s	7,675.2 s	+859.9	100 %	

Two instances carry 92 % of the entire tier gap, and on both Tactix performs roughly twice Kissat's conflicts. That is search trajectory, not the same work taking longer, so no amount of unit-cost work reaches it. The other thirty-eight ranks net to +67 s between them.

The +427.9 s of inprocessing gap is therefore a different target from the 1.112× headline. It is real, it is unit cost, and it is spread across every rank — but closing the two trajectory instances alone would take the tier to roughly 1.009×. Which of the two goals is being served should be decided before the next repair is chosen.

One explanation for those two ranks was raised and refuted the same night. Rank 17 learns clauses at glue 26.76 against Kissat's 15.05, and Kissat spends 11 % of Rank 21 shrinking clauses, so weaker learned-clause quality would inflate propagations and conflicts together. Measuring the ranks that actually carry the gap kills it: quality is matched on Ranks 21, 22 and 34, and on Rank 31 Tactix's clauses are *better* than Kissat's (39.14 literals at glue 23.15 against 47.52 and 26.63). Rank 17 is an isolated instance. Kissat prints no average learned size, so this needed instrumenting `kissat_update_learned`, with its verdict and `clauses_learned` checked against the pristine build first.

8 Where this leaves the work

The 428 s is diffuse. Within each rule the rule's own code is a minority of its time — on Rank 21 `bve.cpp` is 9.3 s of BVE's 35.2 s — and the shared machinery it charges into is structurally faithful to Kissat. Two shared paths carry measured gaps and neither is a local fix: dense index construction is 4.14 s against roughly 1.4 s and is call-count driven, since Tactix rebuilds once per rule application where Kissat connects once per dense phase and lets the rules share it; and collection is about 5.2 s against 1.95 s.

The sharpest single number remains BVE's: on Rank 21 it performs 97.8 M resolutions where Kissat performs 180.8 M, eliminates 3.5 % more variables from a third of the attempts, and is 3.2× slower per resolution.

Two documentation budgets are over: `PROGRESS.md` at 3,800 lines against 350 and `docs/implementation-notes.md` at 591 against 560. Compaction was not requested.

Evidence is in `benchmarks/reports/2026-09-02-unit-cost-and-fidelity-stack.md`, sections 3.12 to 3.25, with campaigns `5010` through `5064`.